



Figure 1: Dart's objectives

choice for you. It can be downloaded from <https://dartpad.dartlang.org>. The DartPad interface is plain and simple. Write your code and click 'Run' to execute the code. The output is also shown within the DartPad window itself, as shown in Figure 3.

DartPad introduces users to the world of Dart programming. The support for libraries in DartPad is restricted to the basic level. To include various libraries, users can have a local installation of the Dart SDK.

A simple Dart program

Shown below is a simple Dart program with a user defined function:

```
// Define a function.
printOSFYIssueNumber(num aNumber) {
  print('Welcome to the OSFY issue Number $aNumber.');//
  Print to console.
}

// This is where the app starts executing.
main() {
  var issuenumber = 12; // Declare and initialize a variable.
  printOSFYIssueNumber(issuenumber); // Call a function.
}
```

`main()` is the function that the application execution starts from. This sample program initialises a variable and calls a user defined function `printOSFYIssueNumber()`. The function takes one numeric argument. In the function, the type of the argument is 'num'. In Dart, 'num' denotes the 'number' type. There are two sub-types to 'num'. These are 'int' and 'double'.

As you might have observed, the function definition doesn't require any specific keyword. It merely involves the use of the function name and the optional arguments. The object-oriented attribute has been given emphasis in the design of the Dart programming language. The functions are objects in Dart. They belong to a type called 'Function'. What this implies is that the functions can also be assigned to other variables, and they can be supplied as arguments to other functions.

The comments are prefixed with `//`. The 'print()' function is used to output the value to the user.

The Dart programming language supports various data

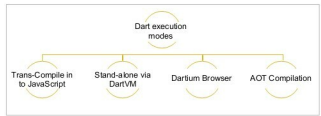


Figure 2: Dart execution modes

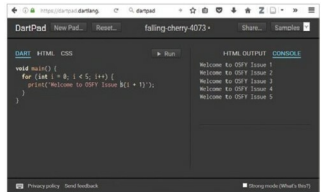


Figure 3: DartPad – execute your Dart programs

types, as listed below:

- Numbers
- Strings: These are sequences of UTF-16 code units. Strings can be provided with either single quotes or double quotes.
- Booleans: These are provided with the 'bool' keyword.
- Lists: These are used to specify arrays, for example, `var list = [100,200,300]`
- Maps: These are used to associate keys with values. For example:

```
var OSFYThemes = {
  // Keys   Values
  'Jan' : 'Mobile Computing',
  'Feb' : 'Networking',
  'Mar' : 'Programming Lang'
};
```

- Runes: In Dart, runes are UTF-32 code points of a string. As stated earlier, Dart strings are UTF -16. For expressing the 32-bit Unicode values, a specific syntax is adopted. An example of how 'runes' are used follows:

```
main() {
  var clapping = '\u{1f44f}';
  print(clapping);
  print(clapping.codeUnits);
  print(clapping.runes.toList());
}
```

- Symbols: The 'Symbol' object in Dart is used to represent